Background:

Trading may be viewed as consisting of two components:

1. What is the fair price
2. How to trade (e.g. accounting for slippage)
    a. Risk management
    b. Non-stationarity in the alpha over time (need to retrain)

Most existing work in LLMs for trading/alpha mining focuses on 1.

Potential Research Gaps:

1. Improve Evaluation: ground truth alpha introduced into the financial data and varying the difficulty of discovery
    a. Varying the appearance of the alpha by varying the market participants (bots)
    b. Online trading simulator
    c. Test on live future market data = verified reward
    d.
2. Requires changes to what is considered best practices from RLVR for math:
    a. Proxy vs gold truth gap: math RLVR recipes are tuned for a world where the reward is exact and exploration is the bottleneck. With a statistical verifier, reward fidelity is the bottleneck, so training should be conservative.
    b. Data is highly noisy, want to avoid fitting to noise: https://arxiv.org/abs/2505.17989
    c. All rollouts in a group share the same data slice, so their noise is correlated and the effective batch is smaller
    d. Apply techniques from overoptimization in RLHF since there is only proxy reward https://arxiv.org/abs/2210.10760
3. Goal of trading agent:
    a. Maximize high precision (AI end to end)
    b. Maximize high recall (AI -> human quant)
    c. How will quants use this?
        i. Manager of AI (research director)
        ii. Claude build strategies
        iii. Issue is that claude results in “obvious” ideas
4. Advantages specific to LLMs in trading:
    a. Ability to identify and chain together multiple trades for arbitrage (a la chaining security exploits)
5. More generally, the problem of noisy time series data + pattern discovery + avoiding overfitting also shows up in AI4Science

Other observations:

a. Trading data is lower dimensional than text (numbers)? This may create some room for doing other stuff

Some Existing Works:

1. Alpha-R1: RL trains a daily factor selector https://arxiv.org/html/2512.23515?utm_source=chatgpt.com
2. Trading-R1: RL trains an asset-day recommendation and thesis https://arxiv.org/html/2509.11420v1
3. AlphaAgentEvo: RL trains how to improve a formula through tool calls https://openreview.net/forum?id=lNmZrawUMu
4. AlphaPilot: RL trains a daily, tool-using factor-pool builder https://arxiv.org/html/2610.15488v1
5. AQuA: Recursively Self-Improving Quantitative Trading Research Agents (autoresearch like work) https://arxiv.org/abs/2608.12841

Starting Point 11/04:

High level goal: leverage as much of existing code as possible, start with a relatively complete repo:

1. Swap out eval for synthetic eval with groundtruth
2. Swap out their GRPO algorithm
    a. See if improvements can be made e.g. with RL techniques that have been designed for forecasting (https://arxiv.org/abs/2505.17989)

Starting Repo:

Shortlist: most complete, easiest to modify

Rank	Repo	Runnable out of the box?	Swap the eval	Swap the GRPO algorithm	Verdict

1	**QuantLLM/QuantEvolver**(MIT)	**Mostly.** The DSL, evaluator, task bank, verl dataset, reward bridge and launcher are all included, and its tests pass here. You bring OHLCV parquet files and a base model.	**One class.** Replace `QuantRewardBridge`; it already returns `reward_extra_info`, so you can log gold next to proxy.	**One dict** of verl overrides in `rft/verl_main.py`.	**Start here (single-turn).**
2	**AlphaAgentEvo official supplementary** (your zip)	**No.** Key pieces are missing (below).	Easy in principle: the whole reward is `FactorTool.calc_reward`.	verl command-line flags (`algorithm.adv_estimator`, KL settings, and so on).	**Use as the multi-turn templat**

					e, not as a base.
3	FinStep-AI/Alpha-R1	Partly. Config and reward are included, but there's no script that builds the RL training prompts.	`training/reward.py` via `custom_reward_function`	Cleanest: stock `main_ppo` with a YAML that already exposes `norm_adv_by_std_in_grpo`	Reference only. It selects factors rather than generating them, and is configured for 32 GPUs.
4	hongha5192-bit/AlphaAgentEvo (unofficial)	No; still being set up, per its own status file	Tool-level	CLI flags	Skip, unless you want their notes on getting the multi-turn

setup to run.

What's in the AlphaAgentEvo supplementary

Included

- **A vendored verl copy** (version file reads `0.4.1.dev`). It differs from the v0.4.1 tag in many places, so it's a development snapshot rather than a release.
- **Their actual changes are small:**
  - `verl/tools/factor_tool.py`: the tool plus the full hierarchical reward;
  - `factor_ast.py`: the similarity computation;
  - `reward_score/factor.py`: a format-only score that ends up unused;
  - a 5-line patch to `workers/reward_manager/naive.py` that overrides the score with the tool's reward.
- **The launch script:** Qwen3-4B-Thinking-2507, GRPO, group size n=3, 2 turns, learning rate 1e-6, KL 0.001, 150 steps, 10 GPUs, sglang rollout.
- **Data:** AlphaEvo500 seed prompts as parquet, 300 train / 30 val / 100 test. The paper says 350/50/100.

Missing

- **`fast_backtest_api`**, the backtest engine that `api_server_fast.py` imports.
- The **`expression_manager`** package: parser, function library, and the code template.
- **All market data.** The seeds use Tushare chip-distribution and order-flow fields such as `$cost_95pct` and `$weight_avg`.
- **The README describes a `backtest_api/` directory that isn't in the zip.**

So the evaluator can't run as shipped. For your purposes that matters less than it sounds, since you're replacing the evaluator anyway. The training loop and reward are present.

What the code confirms about their method

- **The tool's backtest window is hard-coded to 2023-01-01 → 2024-01-01**, and the reward is computed from the same feedback the agent sees. So the agent is rewarded on exactly the data it queries.
- **The performance term uses `best_metric`,** the maximum over all attempts, and is multiplied by a streak bonus for consecutive improvements. This is the improvement-chasing reward we flagged.
- **Possible bug:** the reward divides `(exploration + consistency)` by `base_reward = 0.1·calls − 0.2·failures`. That can be zero or negative, for

example 2 calls with 1 failure gives 0. As written, this would divide by zero or flip the sign.

Recommendation

- **Single-turn core study:** fork **QuantEvolver**.
  - Turn off its archive, novelty and relative-reward shaping.
  - Drop in your ground-truth evaluator class.
  - Make the `algorithm` and `actor` overrides configurable.
- **Multi-turn arm:** start from the **AlphaAgentEvo supplementary**, but point `FactorTool._run_backtest` at your evaluator instead of their missing API. That's the only call site. Keep their `calc_reward` as the "their algorithm" baseline, then swap in the terminal-holdout reward.

QuantEvolver:

QuantEvolver turns factor mining into GRPO over one-line formulas in its DSL, using verl. The two places you'll change are well separated:

- **The eval** is a single Python class (the "reward bridge") that verl calls on each batch of rollouts.
- **The training algorithm** is a dict of verl overrides, plus verl's registry of advantage estimators.

Below I give the normal workflow, then each swap with code.

What I checked:

- The repo's tests pass.

- verl **v0.4.1** has every API that `verl_main.py` calls (`create_rl_dataset`, `create_rl_sampler`, `register_adv_est`, the group `uid` being set before the reward is computed).
- QuantEvolver's bundled base config is a subset of v0.4.1's.

What I didn't check: I had no GPU, so I didn't run training.

1. How it's laid out and how you use it

Pipeline and files

Stage	Command	Code
Describe the scenario (optional, needs an LLM API)	`quantevolver refine-scenario "..."`	`scenarios/`
Validate and score seed factors	`quantevolver validate-seeds examples/seed_candidates.yaml -o runs/seed_evaluations.yaml`	`seeds/`
Build the task bank (seeds × time splits)	`quantevolver build-task-bank runs/seed_evaluations.yaml -c configs/example_seed_pipeline.yaml -o runs/task_bank.yaml`	`rft/task_bank.py`
Check one formula	`quantevolver rft-eval-expr '<expr>' --symbol ASSET --data-dir data ...`	`dsl/`, `evaluation/`
Train	`quantevolver rft-launch configs/example_rft_pure_v`	`rft/pure_verl_backend.py` → `rft/verl_main.py`

```
erl.yaml [--dry-run
--print-command]
```

What happens during training

1. `rft-launch` writes a shell script that activates a conda env (`train_conda_env`) and runs `python -m quant_evolver.rft.verl_main --config ...`.
2. `verl_main.build_verl_config()` writes `train.parquet` and `val.parquet`. Each row is one prompt for one task, with `extra_info`holding `task_id`, `seed_expr`, `time_split`, `start_date` and `end_date`.
3. It merges `configs/verl_ppo_trainer_base.yaml` with a hard-coded overrides dict.
4. It builds `RayPPOTrainer(..., reward_fn=QuantRewardBridge(...))` and calls `fit()`.
5. The model must output `<expr>...</expr>`. The bridge extracts the formula, scores it on that row's window, and writes the scalar reward on the last response token.

Setup

```shell
pip install -e '.[rft,backtest]'
# Separate conda env for training: torch, vLLM, and verl pinned to
v0.4.1
```

- **Data format:** one parquet file per symbol under `data_dir`, named by `data_file_template` (default `{symbol_safe}_1m.parquet`), with columns `open, high, low, close, volume` and a DatetimeIndex. The bundled evaluator resamples to `bar_minutes`, so it's built for intraday bars.
- **Cross-sectional mode:** set `reward_kind: cross_sectional_rankic` and `universe_symbols: [...]` in the `rft:` config.

2. Swapping the eval

The contract

verl calls `reward_fn(data, return_dict=True)`. Your function must return:

- `reward_tensor`, of shape (batch × response length), with each reward placed on the last valid response token;

- `reward_extra_info`, a dict of lists, each the same length as the batch.

verl writes the extra info into its rollout dumps (`trainer.rollout_data_dir`, which QuantEvolver already sets). That's where gold, standard error and per-fold proxies get logged, without the trainer ever seeing them.

Step 1: write your reward class

Python

```
# quant_evolver/rft/gt_reward.py
import numpy as np, torch
from quant_evolver.dsl.compiler import extract_expr

class GroundTruthReward:
    def __init__(self, tokenizer, evaluator, invalid_score=-1.0):
        self.tok, self.ev, self.invalid = tokenizer, evaluator,
invalid_score

    def _extra(self, data, key, default=None):
        ex = data.non_tensor_batch.get("extra_info")
        return [ (e or {}).get(key, default) for e in ex ]

    def __call__(self, data, return_dict=False):
        R = torch.zeros_like(data.batch["responses"],
dtype=torch.float32)
        plen = data.batch["prompts"].shape[-1]
        rlen = data.batch["attention_mask"][:, plen:].sum(dim=1)
        texts = self.tok.batch_decode(data.batch["responses"],
skip_special_tokens=True)
        starts, ends = self._extra(data, "start_date"),
self._extra(data, "end_date")
        extra = {k: [] for k in ["expr", "proxy", "proxy_se",
"gold", "valid"]}
        scores = []
        for i, t in enumerate(texts):
            e = extract_expr(t)
            out = self.ev(e, starts[i], ends[i]) if e else None   #
your evaluator
            ok = out is not None
```

```
            s = out["proxy"] if ok else self.invalid
            scores.append(s)
            extra["expr"].append(e);
extra["valid"].append(float(ok))
            extra["proxy"].append(s)
            extra["proxy_se"].append(out["se"] if ok else 0.0)
            extra["gold"].append(out["gold"] if ok else np.nan)   #
logged only, never used for training
        R[torch.arange(len(scores)), rlen - 1] =
torch.tensor(scores, dtype=torch.float32)
        return {"reward_tensor": R, "reward_extra_info": extra} if
return_dict else R
```

Your evaluator `ev(expr, start, end)` returns `{"proxy": ..., "se": ..., "gold": ...}`. Everything about the evaluation lives behind it: planted signals, folds, the lower-confidence-bound reward, costs.

Step 2: wire it in (about 10 lines in `verl_main.py`)

Python

```python
# rft/config.py → add two fields to RFTConfig
reward_impl: str = "quant_evolver"        # or "ground_truth"
reward_kwargs: dict = field(default_factory=dict)

# rft/verl_main.py → in run(), where reward_fn is built
if rft.reward_impl == "ground_truth":
    from quant_evolver.rft.gt_reward import GroundTruthReward
    from my_eval import build_evaluator            # your package
    reward_fn = GroundTruthReward(tokenizer,
build_evaluator(**rft.reward_kwargs))
else:
    reward_fn = QuantRewardBridge(...)             # unchanged
```

Pass a *separate* instance as `val_reward_fn` if validation should score on a held-out era rather than the training windows.

Step 3: control the data slices through the task bank

- Each prompt row has its own `start_date`/`end_date`, taken from `task_bank.time_splits`. All G rollouts of a prompt therefore share one slice, which gives you **common random numbers for free**.
- Define many slices so that different prompts get different windows.

Things to change at the same time

1. **Remove the dates from the prompt.** `rft/prompts.py` prints `Time split: name (start to end)` (around line 146). That tells the model which period it's being scored on, a contamination channel. Replace it with an anonymous ID.
2. **Keep shaping off unless you're ablating it.** The bridge's archives only activate when their flags or paths are set:
    - `relative_reward` (an improvement bonus against the seed),
    - `diversity_reward` + `family_archive_path`,
    - `residual_response_reward`,
    - `xsec_behavior_diversity`,
    - `novelty_archive_path`.
3. Your own class bypasses all of these anyway.
4. **Decide which DSL to use.** If the model keeps writing QuantEvolver's DSL, your evaluator has to interpret that grammar (the operator list is `ALL_OPS` in `dsl/compiler.py`). The bundled interpreter, `dsl/evaluator.py`, is a per-bar Python loop, too slow for a 500-stock × 2,500-day panel. Either write a vectorized interpreter for the same grammar, or switch the prompt (`rft/prompts.py`) to your own causal DSL and parse that instead.

3. Swapping the training algorithm

Level 0: config only

All overrides currently sit in a hard-coded dict in `build_verl_config()`. `RFTConfig` already has an `extra_overrides: list[str]` field, but nothing reads it. Wire it up so every algorithm change becomes a YAML line:

```python
# end of build_verl_config(), before OmegaConf.resolve
cfg = OmegaConf.merge(base, overrides,
OmegaConf.from_dotlist(rft.extra_overrides))
```

```
None
rft:
  extra_overrides:
    - algorithm.adv_estimator=grpo              # grpo | rloo |
remax | reinforce_plus_plus_baseline | opo | grpo_passk
    - algorithm.norm_adv_by_std_in_grpo=False   # Dr. GRPO / Turtel
"Modified GRPO"
    - actor_rollout_ref.actor.use_kl_loss=True
    - actor_rollout_ref.actor.kl_loss_coef=0.01
    - actor_rollout_ref.actor.clip_ratio_low=0.2
    - actor_rollout_ref.actor.clip_ratio_high=0.2   # symmetric: no
clip-higher
    - actor_rollout_ref.actor.entropy_coeff=0
    - actor_rollout_ref.actor.ppo_epochs=1
    - actor_rollout_ref.rollout.n=16
    - trainer.total_epochs=1                    # single pass
```

How the modifications from the experiment design map onto this:

Modification	Setting
B1: no std normalization	`algorithm.norm_adv_by_std_in_grpo=False`
B2: RLOO / ReMax baselines	`algorithm.adv_estimator=rloo` or `remax`. In v0.4.1, ReMax runs an extra greedy rollout to get its baseline.
C1: single pass	`trainer.total_epochs=1`, `ppo_epochs=1`, and enough distinct task rows (`train_prompt_rows`) that no slice is reused
C2: KL sweep	`use_kl_loss`, `kl_loss_coef`
C5: no clip-higher, no entropy bonus	Clip ratios set equal; `entropy_coeff=0`
Group size	`rollout.n` (QuantEvolver's `rollout_n` field)
C3: early stopping	`trainer.test_freq` plus a held-out `val_reward_fn`; select the checkpoint on the val proxy

Level 1: custom advantage estimators

In verl v0.4.1, every estimator except GAE and GRPO is looked up in a registry and called with `token_level_rewards`, `response_mask`, `index` (the group `uid`) and `config`:

```python
# my_algos.py — import this inside verl_main.run() before building the trainer
import torch
from collections import defaultdict
from verl.trainer.ppo.core_algos import register_adv_est

@register_adv_est("rank_grpo")                    # B5: rank-based advantages within each group
def rank_grpo(token_level_rewards, response_mask, index, config=None, **kw):
    s = token_level_rewards.sum(-1)
    groups = defaultdict(list)
    for i, g in enumerate(index): groups[g].append(i)
    adv = torch.zeros_like(s)
    for idx in groups.values():
        r = s[idx].argsort().argsort().float()
        adv[idx] = (r - r.mean()) / max(len(idx) - 1, 1)
    adv = adv.unsqueeze(-1) * response_mask
    return adv, adv
```

Then set `algorithm.adv_estimator=rank_grpo`.

Estimators that need the evaluator's standard error (B3 SNR shrinkage, B6 significance filter)

The registry interface never sees `reward_extra_info`, so there are two options.

(a) No verl patch (recommended). The `uid` is already in the batch when the reward is computed, so do the shrinkage inside your reward class. Emit $r_i' = \bar{r} + s_g \cdot (r_i - \bar{r})$, where

$$s_g = \max!\left(0,\ 1 - \overline{se^2} / s^2_{group}\right)$$

with $\bar{r}$ the group mean, $\overline{se^2}$ the mean of the evaluator's squared standard errors, and s^2_{group} the observed reward variance within the group. Then train with `adv_estimator=grpo` and `norm_adv_by_std_in_grpo=False`. Group-centering r' gives exactly $s_g(r_i - \bar{r})$.

The significance filter is the special case $s_g \in 0, 1$. Log the raw proxy as well so the curves stay interpretable.

(b) A small verl patch. In `ray_trainer.compute_advantage`, also pass `data.non_tensor_batch` (where the extra info lives) into `adv_kwargs`. Cleaner, but you're then maintaining a fork of verl.

Sanity checklist before real runs

1. `rft-eval-expr` on a known planted formula returns the expected proxy and gold.
2. `rft-launch ... --dry-run --print-command` produces a config containing your overrides.
3. Run a tiny training job: 1 GPU, `train_batch_size=4`, `rollout_n=4`. Confirm the rollout dump contains `gold` and `proxy_se`, and that rewards vary within groups.
4. Run the baseline with QuantEvolver's own bridge and shaping turned on once. That is the "their algorithm" reference before you start ablating.

GRPO modifications to try, grouped by cost to implement:

Config-only (verl flags)

#	Change	Expected effect	Basis
B1	Disable std normalization (`norm_adv_by_std_in_grpo=False`)	Stops groups whose spread is pure noise from producing full-size updates	Dr. GRPO; Turtel et al.'s Modified GRPO (standard GRPO drove overconfidence and collapse under noisy rewards)
B2	`adv_estimator=remax` or `rloo`	Lower-variance baselines; ReMax was Turtel's best	Turtel et al.
C1	Single pass: 1 total epoch, a unique slice per prompt, ppo_epochs=1	Less memorization of the training history. Turtel found repeated epochs overfit severely.	Turtel et al.
C2	KL sweep, e.g. {0, 1e-3, 1e-2}, with the KL loss on	KL as shrinkage toward the prior	Coste (small KL plus conservative objective); Gao (KL behaves like early stopping)
C5	Clip symmetric, no clip-higher, no entropy bonus	You want pass@1 sharpening on gold, not preserved pass@k	Analysis earlier in this thread
—	Group size n ∈ {8, 16}	Larger groups give better within-group comparisons under noise	—

Inside the reward function (your eval code)

#	Change	Expected effect
A2	Lower-confidence-bound reward: mean − λ·sd across purged folds (uncertainty-weighted), or the minimum across folds (worst case)	Pessimism against the winner's curse; folds play the role of ensemble members (Coste et al.)

A3	Residualize against baseline factors and the existing library	Rewards marginal value only
A4	Abstain action with reward 0	Makes precision learnable; placebo worlds test it
A7	Reward net of costs and turnover	Aligns the proxy with deployable value
B5'	Winsorize the reward	Robustness to fat tails

Small code: one custom advantage estimator

#	Change	Expected effect
B3	**SNR shrinkage:** $A_i = (r_i - \bar{r}) \cdot \max(0, 1 - \overline{se^2}/s^2_{group})$, where $\overline{se^2}$ is the mean of `proxy_se²` and s^2_{group} the observed reward variance within the group	Updates shrink toward zero when within-group spread is mostly evaluator noise. **The most novel item**, and only possible because a statistical verifier knows its own error.
B5	Rank-based advantages within each group	Robust to outliers
B6	Zero out groups whose spread isn't significant relative to `proxy_se`	Replaces DAPO's zero-variance filter, which never fires on continuous noisy rewards

Protocol

#	Change	Expected effect
C3	Early stopping on a held-out era's proxy (pessimistic, in the style of Karwowski et al.)	Stops near the gold peak without access to gold
D1	Median or ensemble sampling at inference, plus ensembles across seeds	Lower variance in what gets deployed (Turtel's 7-run ensemble)

Multi-turn only. Requires the AlphaAgentEvo code or the verl agent-loop fallback.

- **A1:** terminal reward on a sealed holdout specific to each episode, versus AlphaAgentEvo's best-so-far and streak rewards. This is the single most informative multi-turn ablation.
- **A5:** Ladder/Thresholdout-filtered validation feedback to the agent.

Order and hypotheses

1. **B1 + B2.** Cheap, and grounded in Turtel's results. *Hypothesis:* stabilizes training, with a modest effect on gold.
2. **A2 + CRN** (the free shared slice per prompt). *Hypothesis:* the largest improvement in the gold-versus-proxy gap.
3. **C1.** *Hypothesis:* repeated epochs show proxy rising while gold falls.
4. **B3.** *Hypothesis:* beats B1 at low SNR; no difference at high SNR.
5. **C2 + C3.** KL sweep plus early stopping.
6. **A4.** *Hypothesis:* placebo FDR drops with little cost to gold on worlds with real signal.

Then combine the winners and run the combination across SNR levels.

Budget. About 10 single-change runs plus 2–3 combinations, × 3 seeds, × 2–3 SNR levels: roughly 100 short runs. Most changes are flags or reward-file edits, so the engineering cost is concentrated in P1 (generation output plus your eval) and B3.